



Topics Covered — Day 4

- 1. Primary Key — Definition, Syntax, Examples, Rules, Common Errors
- 2. Foreign Key — Definition, Syntax, Examples, Rules, Common Errors
- 3. Constraints — NOT NULL, UNIQUE, DEFAULT, CHECK
- 4. Real-Time Interview Questions & Answers
- 5. Quick Revision Cheat Sheet

1. Primary Key

Definition

A Primary Key is a column (or a combination of columns) in a table that UNIQUELY identifies every row.

Think of it like an Aadhaar number — no two people can have the same Aadhaar!

Rules: (1) Must be UNIQUE — no two rows can have the same value.

(2) Cannot be NULL — every row must have a value.

(3) A table can have only ONE Primary Key.

1.1 Syntax

Method 1: Define Primary Key inside CREATE TABLE

```
CREATE TABLE table_name
(
    column1 datatype PRIMARY KEY,
    column2 datatype,
    column3 datatype
);
```

Method 2: Define Primary Key at the bottom (Recommended for multi-column PK)

```
CREATE TABLE table_name
(
    column1 datatype,
    column2 datatype,
    CONSTRAINT pk_name PRIMARY KEY (column1)
```

```
);
```

Method 3: Add Primary Key to an existing table using ALTER TABLE

```
ALTER TABLE table_name  
ADD CONSTRAINT pk_name PRIMARY KEY (column_name);
```

1.2 Example

Create a Students table where studentid is the Primary Key:

```
-- Create a Students table with Primary Key  
CREATE TABLE flmstudents  
(  
    studentid    INT          PRIMARY KEY,  
    studentname  VARCHAR(100),  
    emailid      VARCHAR(100),  
    address      NVARCHAR(250)  
);  
  
-- Insert data  
INSERT INTO flmstudents VALUES (1, 'Rahul Sharma', 'rahul@gmail.com',  
'Hyderabad');  
INSERT INTO flmstudents VALUES (2, 'Priya Singh', 'priya@gmail.com', 'Mumbai');  
  
-- This will FAIL -- duplicate primary key!  
INSERT INTO flmstudents VALUES (1, 'Amit Kumar', 'amit@gmail.com', 'Delhi');  
-- Error: Violation of PRIMARY KEY constraint. Cannot insert duplicate key.
```

Output after successful inserts:

studentid (PK)	studentname	emailid	
1	Rahul Sharma	rahul@gmail.com	Hyderabad
2	Priya Singh	priya@gmail.com	Mumbai

1.3 Rules

- A table can have only ONE Primary Key.
- The Primary Key value must be UNIQUE for every row — no duplicates allowed.
- Primary Key columns cannot hold NULL values.
- You can create a composite Primary Key using two or more columns (e.g., PRIMARY KEY (col1, col2)).
- Primary Key automatically creates a UNIQUE index on the column(s).

Common Errors

Error 1: Inserting a duplicate value — 'Violation of PRIMARY KEY constraint pk_flmstudents.'

Fix: Use a different (unique) value for the primary key column.

Error 2: Inserting NULL into a Primary Key column — 'Cannot insert the value NULL into column studentid.'

Fix: Always provide a non-null value for the primary key.

Error 3: Creating two Primary Keys on same table — 'Cannot add two PRIMARY KEY constraints to table.'

Fix: A table can have only ONE primary key (but it can span multiple columns).

Quick Tip

Best Practice: Always name your Primary Key constraint — `CONSTRAINT pk_students PRIMARY KEY (studentid)`.

This makes it easy to identify and drop/modify constraints later.

Use INT or BIGINT for primary key columns — they are fast and auto-incrementable.

2. Foreign Key

Definition

A Foreign Key is a column in one table that REFERS TO the Primary Key of another table. It creates a LINK (relationship) between two tables.

Example: An Orders table has a customerid column that refers to the customerid (PK) in the Customers table.

This ensures you cannot add an order for a customer who does not exist — this is called Referential Integrity.

2.1 Syntax

Method 1: Define Foreign Key inside CREATE TABLE

```
CREATE TABLE child_table
(
    column1 datatype,
    fk_column datatype,
    CONSTRAINT fk_name FOREIGN KEY (fk_column)
        REFERENCES parent_table(pk_column)
);
```

Method 2: Add Foreign Key to an existing table using ALTER TABLE

```
ALTER TABLE child_table
ADD CONSTRAINT fk_name FOREIGN KEY (fk_column)
    REFERENCES parent_table(pk_column);
```

2.2 Example

Step 1: Create the Parent table (Courses) with a Primary Key:

```
-- Parent table
CREATE TABLE flmcourses
(
    courseid    INT          PRIMARY KEY,
    coursename  VARCHAR(100)
);

INSERT INTO flmcourses VALUES (101, 'SQL Fundamentals');
INSERT INTO flmcourses VALUES (102, 'Python Basics');
INSERT INTO flmcourses VALUES (103, 'Data Analysis');
```

Step 2: Create the Child table (Enrollments) with a Foreign Key:

```
-- Child table - courseid must exist in flmcourses
CREATE TABLE flmenrollments
(
    enrollid    INT    PRIMARY KEY,
    studentname VARCHAR(100),
    courseid    INT,
    CONSTRAINT fk_course FOREIGN KEY (courseid)
        REFERENCES flmcourses(courseid)
);

-- Valid insert - courseid 101 exists in flmcourses
INSERT INTO flmenrollments VALUES (1, 'Rahul Sharma', 101);

-- INVALID - courseid 999 does NOT exist in flmcourses
INSERT INTO flmenrollments VALUES (2, 'Priya Singh', 999);

-- Error: The INSERT statement conflicted with the FOREIGN KEY constraint.
```

Relationship Diagram:



2.3 Rules

- The Foreign Key value **MUST** match an existing Primary Key value in the parent table — or be NULL.
- You cannot delete a row from the parent table if a child row references it (unless you set ON DELETE CASCADE).
- The data types of the FK column and PK column must match.
- A table can have **MULTIPLE** Foreign Keys (referencing different parent tables).
- Always create the Parent table **BEFORE** the Child table.

Common Errors

Error 1: Inserting a value in FK column that doesn't exist in parent table.

'The INSERT statement conflicted with the FOREIGN KEY constraint.'

Fix: Make sure the value exists in the parent table first.

Error 2: Trying to DROP the parent table while the child table still references it.

'Cannot drop table because it is being referenced by a FOREIGN KEY constraint.'

Fix: Drop the child table first, or drop the FK constraint, then drop the parent.

Error 3: Data type mismatch between FK and PK columns.

Fix: Both FK and PK columns must have the same data type (e.g., both INT).

Quick Tip

ON DELETE CASCADE: Automatically deletes child rows when the parent row is deleted.

Syntax: REFERENCES parent_table(pk_column) ON DELETE CASCADE

ON UPDATE CASCADE: Automatically updates child FK values when the parent PK changes.

3. SQL Constraints

Constraints are rules applied to columns to control the kind of data that can be stored. They help maintain data accuracy and integrity.

Constraint	Purpose	Example
NOT NULL	Column must always have a value	<code>studentname VARCHAR(100) NOT NULL</code>
UNIQUE	All values in the column must be different	<code>emailid VARCHAR(100) UNIQUE</code>
DEFAULT	Provides a default value if none is given	<code>city NVARCHAR(100) DEFAULT 'Hyderabad'</code>
CHECK	Ensures values satisfy a condition	<code>age INT CHECK (age >= 18)</code>
PRIMARY KEY	Uniquely identifies each row	<code>studentid INT PRIMARY KEY</code>
FOREIGN KEY	Links to another table's Primary Key	<code>FOREIGN KEY (courseid) REFERENCES ...</code>

3.1 NOT NULL Constraint

Definition

NOT NULL ensures that a column cannot store a NULL (empty) value.
Use it when a field is mandatory — like a student's name or email.

Syntax

```
CREATE TABLE flmstudents
(
    studentid    INT          NOT NULL,
    studentname  VARCHAR(100) NOT NULL,    -- Required field
    emailid      VARCHAR(100),             -- Optional field (can be NULL)
    address      NVARCHAR(250)
);
```

Example — Error when NULL is inserted

```
-- This will FAIL because studentname is NOT NULL
INSERT INTO flmstudents (studentid, studentname) VALUES (1, NULL);
-- Error: Cannot insert the value NULL into column 'studentname'.

-- This is VALID
INSERT INTO flmstudents (studentid, studentname) VALUES (1, 'Rahul Sharma');
```

- Rule: NOT NULL columns must always receive a value in every INSERT statement.
- You cannot update a NOT NULL column to NULL either.

3.2 UNIQUE Constraint

Definition

UNIQUE ensures all values in a column are different — no duplicates allowed.

Unlike PRIMARY KEY, a UNIQUE column CAN contain NULL values (but only once in most databases).

A table can have MULTIPLE UNIQUE constraints.

Syntax

```
CREATE TABLE flmstudents
(
    studentid    INT           PRIMARY KEY,
    studentname  VARCHAR(100),
    emailid      VARCHAR(100) UNIQUE,    -- No two students can have the same email
    phonenumber  VARCHAR(15)  UNIQUE
);

-- Named UNIQUE constraint (recommended)
CONSTRAINT uq_email UNIQUE (emailid)
```

Example

```
INSERT INTO flmstudents VALUES (1, 'Rahul', 'rahul@gmail.com', '9876543210');
INSERT INTO flmstudents VALUES (2, 'Priya', 'rahul@gmail.com', '9999999999');
-- Error: Violation of UNIQUE KEY constraint 'uq_email'. Cannot insert duplicate
key.
```

- UNIQUE vs PRIMARY KEY: PRIMARY KEY = UNIQUE + NOT NULL. UNIQUE alone allows one NULL.

3.3 DEFAULT Constraint

Definition

DEFAULT assigns a pre-set value to a column when no value is provided during INSERT.

Useful when most records share the same value for a column (e.g., city = 'Hyderabad').

Syntax

```
CREATE TABLE flmstudents
(
    studentid    INT           PRIMARY KEY,
```



```

    studentname VARCHAR(100) NOT NULL,
    city         NVARCHAR(100) DEFAULT 'Hyderabad', -- Default city
    isactive     BIT           DEFAULT 1           -- Default: active
);

```

Example

```

-- city not provided - will use default 'Hyderabad'
INSERT INTO flmstudents (studentid, studentname) VALUES (1, 'Rahul Sharma');

SELECT * FROM flmstudents;

-- Result:
-- studentid | studentname | city       | isactive
-- 1         | Rahul Sharma | Hyderabad | 1

```

- DEFAULT only applies when the column is not mentioned in the INSERT statement.
- If you explicitly insert NULL for a DEFAULT column, NULL is stored (not the default).

3.4 CHECK Constraint

Definition

CHECK validates that the value in a column satisfies a specific condition (logical expression). If the condition is FALSE, the INSERT or UPDATE is rejected.
Example: age must be ≥ 18 , or marks must be between 0 and 100.

Syntax

```

CREATE TABLE flmstudents
(
    studentid    INT           PRIMARY KEY,
    studentname  VARCHAR(100),
    age          INT           CHECK (age >= 18),
    marks        DECIMAL(5,2) CHECK (marks >= 0 AND marks <= 100)
);

-- Named CHECK constraint (recommended)
CONSTRAINT chk_age    CHECK (age >= 18),
CONSTRAINT chk_marks CHECK (marks BETWEEN 0 AND 100)

```

Example

```

INSERT INTO flmstudents VALUES (1, 'Rahul', 17, 85.5);

```

```
-- Error: The INSERT statement conflicted with the CHECK constraint 'chk_age'.
-- Age 17 fails the CHECK (age >= 18) rule.

INSERT INTO flmstudents VALUES (2, 'Priya', 22, 105.0);
-- Error: Marks 105 fails the CHECK (marks <= 100) rule.

-- Valid insert
INSERT INTO flmstudents VALUES (3, 'Amit', 20, 88.5); -- Success!
```

Common Errors

Error: 'The INSERT statement conflicted with the CHECK constraint.'

Fix: Make sure the value satisfies the condition defined in the CHECK constraint.

Tip: Use meaningful constraint names — `CONSTRAINT chk_age CHECK (age >= 18)`

This makes error messages easier to understand.

4. Quick Revision Cheat Sheet

```
-- ===== PRIMARY KEY =====
CREATE TABLE flmstudents
(
    studentid    INT            PRIMARY KEY,  -- inline PK
    studentname  VARCHAR(100) NOT NULL,
    emailid      VARCHAR(100) UNIQUE,
    age          INT            CHECK (age >= 18),
    city         NVARCHAR(100) DEFAULT 'Hyderabad'
);

-- Add PK to existing table
ALTER TABLE flmstudents
ADD CONSTRAINT pk_students PRIMARY KEY (studentid);

-- ===== FOREIGN KEY =====
CREATE TABLE flmenrollments
(
    enrollid     INT PRIMARY KEY,
    studentid    INT,
    courseid     INT,
    CONSTRAINT fk_student FOREIGN KEY (studentid)
        REFERENCES flmstudents(studentid),
    CONSTRAINT fk_course FOREIGN KEY (courseid)
        REFERENCES flmcourses(courseid)
);

-- ===== ALL CONSTRAINTS TOGETHER =====
CREATE TABLE flmemployees
(
    empid        INT            PRIMARY KEY,      -- Unique + Not Null
    empname      VARCHAR(100) NOT NULL,          -- Mandatory
    email        VARCHAR(100) UNIQUE,            -- No duplicates
    salary       DECIMAL(10,2) CHECK (salary > 0), -- Must be positive
    department   VARCHAR(50)  DEFAULT 'General', -- Default dept
    managerid    INT,
    CONSTRAINT fk_manager FOREIGN KEY (managerid) -- FK reference
        REFERENCES flmemployees(empid)
);
```

Constraints at a Glance

Constraint	Allows NULL?	Allows Duplicates?	Per Table Limit
PRIMARY KEY	No	No	Only 1
UNIQUE	Yes (once)	No	Multiple
NOT NULL	No	Yes	Multiple
DEFAULT	Yes	Yes	Multiple
CHECK	Yes	Yes	Multiple
FOREIGN KEY	Yes	Yes	Multiple

5. Real-Time Interview Questions & Answers

These questions are commonly asked in SQL interviews for fresher and junior developer roles. Study them carefully!

5.1 Primary Key Questions

Q: What is a Primary Key? Why is it important?

A: A Primary Key is a column (or set of columns) that uniquely identifies each row in a table. It is important because it prevents duplicate records and makes it easy to locate any specific row. Rules: it must be unique and cannot be NULL. A table can have only one Primary Key.

Q: Can a table have more than one Primary Key?

A: No. A table can have only ONE Primary Key. However, that Primary Key can span multiple columns — this is called a Composite Primary Key. Example: PRIMARY KEY (studentid, courseid) — together they must be unique.

Q: What is a Composite Primary Key?

A: A Composite Primary Key is a Primary Key that consists of TWO or more columns. No single column alone uniquely identifies a row, but the combination does. Example: In an Enrollments table, (studentid, courseid) together can be the Composite PK since a student can enroll in many courses.

Q: What is the difference between PRIMARY KEY and UNIQUE?

A: PRIMARY KEY = UNIQUE + NOT NULL. Primary Key cannot be NULL and only one is allowed per table. UNIQUE allows NULL values (once) and a table can have multiple UNIQUE constraints. Both prevent duplicate values in the column.

Q: Can a Primary Key column have NULL values?

A: No. A Primary Key column must always have a value — NULL is not allowed. This ensures every row can be uniquely identified. If you try to insert NULL into a PK column, you will get an error: 'Cannot insert the value NULL into column.'

5.2 Foreign Key Questions

Q: What is a Foreign Key? Give an example.

A: A Foreign Key is a column in one table (child) that refers to the Primary Key of another table (parent). It enforces Referential Integrity. Example: In an Enrollments table, courseid is a Foreign Key that references courseid in the Courses table. You cannot enroll a student in a course that does not exist.

Q: Can a Foreign Key column contain NULL?

A: Yes. A Foreign Key column CAN contain NULL. NULL means 'no reference' — it indicates the relationship is unknown or not applicable. This is different from a Primary Key, which cannot be NULL.

Q: Can a table have multiple Foreign Keys?

A: Yes. A table can have MULTIPLE Foreign Keys, each referencing a different parent table. Example: An Enrollments table can have a Foreign Key to the Students table AND another Foreign Key to the Courses table.

5.3 Constraints Questions

Q: What are SQL Constraints? Name the types.

A: SQL Constraints are rules applied to columns to control the data that can be stored. Types: (1) NOT NULL — column must have a value. (2) UNIQUE — all values must be different. (3) DEFAULT — provides a default value. (4) CHECK — validates a condition. (5) PRIMARY KEY — unique + not null. (6) FOREIGN KEY — links to another table.

Q: What is the difference between NOT NULL and DEFAULT?

A: NOT NULL means a column must always receive a value — you cannot skip it. DEFAULT provides an automatic value when you do NOT specify the column in INSERT. If you use DEFAULT on a NOT NULL column, you can skip it in INSERT because the default fills it in.

Q: What happens if a CHECK constraint fails?

A: If a CHECK constraint fails, the INSERT or UPDATE statement is rejected with an error: 'The INSERT statement conflicted with the CHECK constraint.' The row is not added to the table. Always verify values match the CHECK condition before inserting.

Q: Can you have a DEFAULT and NOT NULL on the same column?

A: Yes! This is actually a good combination. NOT NULL ensures no empty values exist, and DEFAULT ensures the column always has a fallback value if not specified in INSERT. Example: city NVARCHAR(100) NOT NULL DEFAULT 'Hyderabad' — city is required and defaults to Hyderabad.

5.4 Scenario-Based Questions

Q: You are designing a Students table. Which constraints would you apply?

A: studentid INT PRIMARY KEY — unique identifier. studentname VARCHAR(100) NOT NULL — name is mandatory. emailid VARCHAR(100) UNIQUE — no duplicate emails. age INT CHECK (age >= 5) — age must be valid. city NVARCHAR(100) DEFAULT 'Hyderabad' — defaults to main city. This combination ensures data quality and integrity.

Q: Can you delete a parent table row that has matching child rows?

A: No, by default you cannot. You will get a Foreign Key constraint error. Solutions: (1) Delete the child rows first, then delete the parent row. (2) Use ON DELETE CASCADE — this auto-deletes child rows when parent is deleted. (3) DROP the FK constraint, delete parent, then re-add FK.

Q: What is the order to create tables that have Foreign Key relationships?

A: Always create the PARENT table FIRST, then the CHILD table. This is because the Foreign Key in the child table references the Primary Key in the parent table — the parent must exist before the child can reference it. To DROP them: drop the CHILD table first, then the PARENT.